



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

(SECP38843) SPECIAL TOPIC IN DATA ENGINEERING

SECTION 02

GROUP 3

APACHE APARK TUTORIAL

LECTURER: DR. ARYATI BINTI BAKRI

STUDENT NAME	MATRIC NO
GUI KAH SIN	A23CS0080
SABRINA HENG WEI QI	A23CS0265
WOO CHENG SHUAN	A23CS0283

Table of Contents

1.0 Business Understanding	3
1.1 Introduction	3
1.2 Problem Statement	3
1.3 Business Objectives	4
1.4 Expected Outcome	4
2.0 Data Understanding	5
2.1 Overview of the Raw Data	5
2.2 Data Characteristics and Observations	6
2.3 Relational Data Model	6
3.0 Data Preparation	9
3.1 Environment Setup and Infrastructure Configuration	9
3.2 Raw Data Collection and Extraction	10
3.3 Data Preprocessing	10
3.4 Data Transformation from CSV to Parquet	11
4.0 Modeling	12
4.1 ETL/Spark Architecture	12
4.1.1 Architecture Overview	12
4.1.2 Role of Apache Spark in the architecture	12
4.2 Multidimensional Data Model	13
4.2.1 Star Schema Overview	14
4.2.2 How the Star Schema Works	16
5.0 Evaluation	18
5.1 ETL Pipeline Performance and Data Integrity	18
5.2 Dashboard Visualization and Business Objective Evaluation	18
5.3 Limitations, Improvements and Overall Evaluation	21
6.0 Reflection	22

1.0 Business Understanding

1.1 Introduction

The tutorial, named Creating a Simple ETL Pipeline with Apache Spark, focuses on learning how Apache Spark used to build a complete Extract, Transform, and Load (ETL) pipeline used for large datasets analysis purposes. The lesson shows the process of transforming raw data collected by educational census into a structured star schema model that is useful for Business Intelligence dashboards and analytical reporting.

The source dataset in this tutorial was obtained from the Brazilian National Basic Education Census from 2010 to 2021. It contains information about schools with regard to geographical, structural, economic and educational aspects. The data set is made up of large CSV files with around 370 columns and more than 2GB of data, making it difficult to manage and analyze with traditional tools like spreadsheets or Pandas. As a result, Apache Spark has been chosen as the main data processing technology to efficiently manage large-scale datasets.

The tutorial also covers a number of real-world tools for data engineering like Docker, PostgreSQL, Adminer and Metabase to create a full business intelligence environment. The project's end product is a well-organized database that can be used for dashboards and trend analysis for education and decision making.

1.2 Problem Statement

Companies usually collect large amounts of data, but raw datasets are usually not suitable for analysis. In this tutorial, the dataset used refers to Brazilian census data about education in CSV form that is large, complex and ineffective for querying purposes.

The data set has nearly 370 attributes per file and spans several years of educational data, which makes it difficult to process using traditional methods, as they can be time-consuming and computationally intensive. Trying to load all the data into the application using the standard Python libraries can result in poor performance, high memory use, or even application crashes.

Furthermore, raw CSV data does not support faster analytical queries required in business intelligence systems. For example, decision-making questions cannot be efficiently answered without transforming the data into a structured analytical format.

Therefore, the main problem addressed in this tutorial is how to efficiently process large raw educational datasets and transform them into a star schema model suitable for fast aggregation queries and dashboard visualization.

1.3 Business Objectives

The primary business objective of this tutorial is to build an ETL pipeline using Apache Spark that converts raw educational census data to a more suitable data warehouse structure for analytics and reporting purposes.

There are four main objectives included:

1. Optimization of large-scale data processing
Considering the large sizes of CSV files used in the tutorial and the inefficiency for multiple querying, Apache Spark was chosen to analyze this type of data using lazy evaluation and distributed computing concepts. In the tutorial, we use CSV to Parquet conversion since this format provides data compression, reduces space consumption and faster analysis.
2. Star Schema structure
As another goal, we need to create a star schema database structure that involves both dimension tables and fact tables. This structure improves aggregation of query performance and enables efficient business intelligence analysis.
3. Business Intelligence dashboards creation
The data will be loaded into PostgreSQL and then synced to Metabase where analytical dashboards will be built. For instance, displaying changes in the number of students over time, comparison between regions, and identifying the top performing region.
4. To gain practical data engineering experience
This tutorial is a hands-on project-based learning experience to understand real-world ETL workflows with industry technologies such as Apache Spark, PostgreSQL, Docker and BI tools. The tutorial will enable students to appreciate the potential of turning raw big data into valuable business intelligence.

1.4 Expected Outcome

At the end of this tutorial, a fully functioning ETL pipeline is expected to be created. Raw educational census data will be transformed into a structured star schema database optimized for analytical queries.

The PostgreSQL database will contain populated dimension and fact tables capable of supporting business intelligence dashboards in Metabase. Users will be able to generate visual insights such as yearly student enrollment and regional educational comparisons.

Overall, the tutorial provides an example of how Apache Spark can be used to efficiently transform large raw datasets into meaningful business information for analysis and decision-making.

2.0 Data Understanding

The dataset in this tutorial is sourced from Brazil's National School Census (Censo Escolar). It is published yearly by Brazil's National Institute for Educational Studies and Research (INEP) and openly available on the Brazilian government's open data portal. For this project, we were working with 12 years of data from 2010 to 2021. Data was divided by each year's census into separate ZIP files containing a semicolon-delimited CSV file, encoded in Latin-1 (ISO-8859-1). The full raw dataset amounts to hundreds of millions of records about schools in Brazil.

2.1 Overview of the Raw Data

Any given CSV file stood for a census year and had a row for each school. The data covers an extensive range of attributes about each school's infrastructure, location, administration type and student enrollment numbers. Here's a breakdown of the major categories of attributes in the raw dataset:

Attribute Category	Example Fields	Description
Location	NO_UF, SG_UF, CO_UF, NO_MUNICIPIO, CO_MUNICIPIO	State name, state code, municipality name and code
School	TP_DEPENDENCIA, TP_LOCALIZACAO	Administrative dependency and whether rural or urban
Facilities	IN_AGUA_POTAVEL, IN_BANHEIRO, IN_BIBLIOTECA, IN_INTERNET, IN_COMPUTADOR, IN_ENERGIA_INEXISTENTE, IN_ESGOTO_INEXISTENTE, IN_REFEITORIO, IN_EQUIP_NENHUM	Binary flags indicating whether the school has clean water, bathrooms, library, internet, computers, electricity, sewage, cafeteria or no equipment at all
Enrollment	IN_AGUA_POTAVEL, IN_BANHEIRO, IN_BIBLIOTECA, IN_INTERNET, IN_COMPUTADOR, IN_ENERGIA_INEXISTENTE, IN_ESGOTO_INEXISTENTE, IN_REFEITORIO, IN_EQUIP_NENHUM	Total enrollment figures for basic, infant, elementary and secondary education levels
Race	QT_MAT_BAS_BRANCA, QT_MAT_BAS_PRETA, QT_MAT_BAS_PARDA, QT_MAT_BAS_INDIGENA, QT_MAT_BAS_AMARELA, QT_MAT_BAS_ND	Enrollment counts broken down by self-declared race or ethnicity

Teacher Counts	QT_DOC_BAS, QT_DOC_INF, QT_DOC_FUND, QT_DOC_MED	Number of teachers per education level
Census Year	NU_ANO_CENSO	The year the record belongs to (2010 to 2021)

2.2 Data Characteristics and Observations

During data understanding, several interesting characteristics in the raw CSV files were noticed, including:

- File encoding: All CSVs for one census year used Latin-1 (ISO-8859-1) encoding, due to the presence of Portuguese characters (like accents on letters). This had to be explicitly addressed when ingesting with Apache Spark, as newer versions of PySpark no longer recognized 'latin1' as a charset alias and need the full name 'iso-8859-1' instead.
- Delimiter: The CSVs used semicolon as field delimiter rather than the more common comma, which is a common format for Brazilian government data exports.
- Volume: Each individual yearly CSV file was hundreds of megabytes to over a gigabyte when decompressed. Across the 12 years (2010-2021), the total raw size is on the scale of several gigabytes, making direct SQL loading impractical without a distributed processing framework like Apache Spark.
- Schema consistency: While the field names were mostly consistent over years, not all columns existed in any given yearly file. Differences were automatically dealt with by Apache Spark by inferring the schema when reading all CSVs together.
- Binary flags: Infrastructure attributes like in-internet and in_computador are integers (0 or 1) representing flags, making them ideal for use as dimension table values with a small, fixed set of distinct values.
- Missing values: Some enrollments/teacher count fields have null values for school who didn't report the given education levels. Those nulls were left as is and cast to integers during the transformation step.

2.3 Relational Data Model

Before the data is transformed into a star schema for analytics, it is important to gain an understanding of the relational structure of the source data. The raw Census data is stored in a flat, denormalized form – each row in the CSV contains all attributes of a given school for a census year. This means the same information, like state names, municipality codes and infrastructure flags are shown redundantly across the millions of rows.

The relational data model shown below represents how the raw data relates to the entities we later extracted into our dimension tables. It shows key entities, their attributes (or characteristics) and the relationship between each other:

Entity	Key Attributes	Relationship
School	school_id (PK), loc_id (FK), TP_DEPENDENCIA, TP_LOCALIZACAO, NU_ANO_CENSO	Central entity. One school record per census year. Connected to all other entities. One school belongs to one location.
Location	loc_id (PK), NO_UF, SG_UF, CO_UF, NO_MUNICIPIO, CO_MUNICIPIO	One location can have many school (1 to 0..*). Represents the geographic grouping of schools by state and municipality.
Facilities	fac_id (PK), school_id (FK), IN_BANHEIRO, IN_INTERNET, IN_AGUA_POTAVEL, IN_BIBLIOTECA, IN_COMPUTADOR, IN_REFEITORIO, IN_ENERGIA_INEXISTENTE, IN_ESGOTO_INEXISTENTE, IN_EQUIP_NENHUM	Each school has exactly one facilities record (1 to 1). Each attribute is a binary flag (0 or 1) indicating whether the school has that infrastructure.
Enrollment	enr_id (PK), school_id (FK), QT_MAT_BAS, QT_MAT_INF, QT_MAT_FUND, QT_MAT_MED	Each school has exactly one enrollment record (1 to 1). Stores total student enrollment counts per education level.
Teachers	tea_id (PK), school_id (FK), QT_DOC_BAS, QT_DOC_INF, QT_DOC_FUND, QT_DOC_MED	Each school has exactly one teachers record (1 to 1). Stores total teacher counts per education level.
Race	race_id (PK), school_id (FK), QT_MAT_BAS_BRANCA, QT_MAT_BAS_PRETA, QT_MAT_BAS_PARDA, QT_MAT_BAS_INDIGENA, QT_MAT_BAS_AMARELA, QT_MAT_BAS_ND	Each school has exactly one race record (1 to 1). Stores enrollment counts broken down by self-declared race or ethnicity.

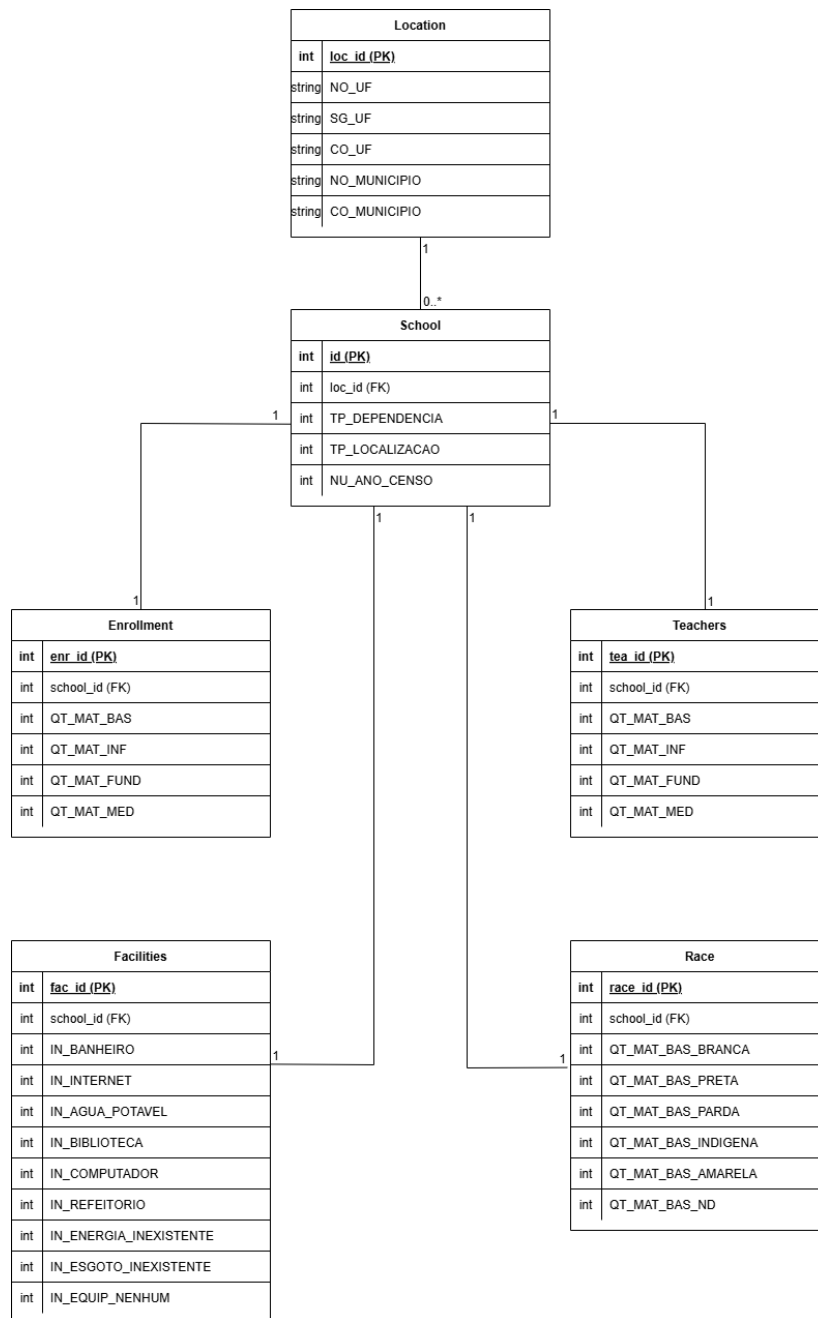


Figure 1: ERD Diagram

The main finding of this relational model is that the raw data is significantly denormalized, which means that every school with those characteristics has the same location name, administrative type or infrastructure status. The flat CSV format is ineffective for analytical queries because of this redundancy. In order to create the star schema mentioned in the Modeling section, the ETL pipeline aims to normalize these repeated characteristics into distinct dimension tables and link them back to the fact table via foreign key connections.

3.0 Data Preparation

One of the most crucial phases of our ETL pipeline project was data preparation. The raw Brazilian Education Census data set was huge, heterogeneous and spread across several yearly CSV files from 2010 to 2021. Apache Spark was chosen as the main processing tool as it is more efficient in processing large-scale datasets compared to traditional tools such as Pandas, which may struggle or crash when handling several gigabytes of data due to memory limitations. The main goal of this phase was to transform 12 raw census files into a clean, structured and analytics-ready dataset before loading it into PostgreSQL for multidimensional analysis.

3.1 Environment Setup and Infrastructure Configuration

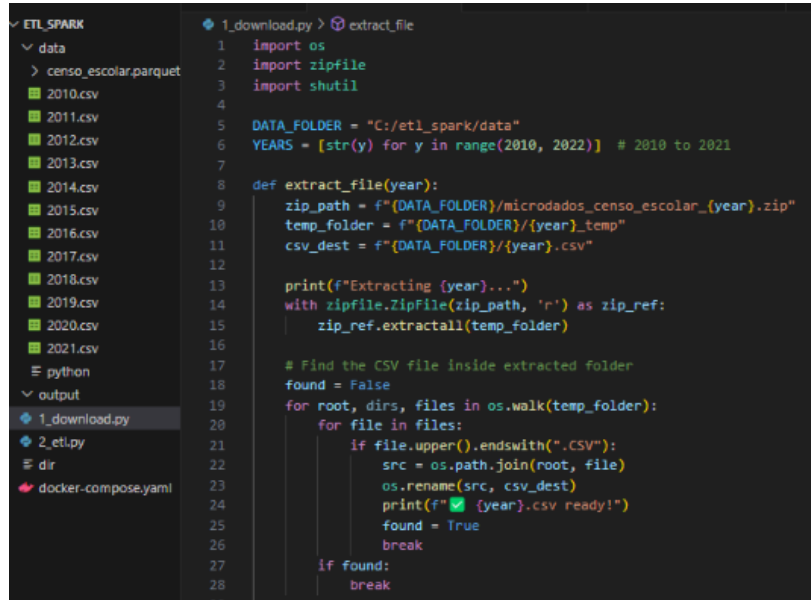
Before starting the ETL process, the necessary environment and supporting tools were installed to make sure Apache Spark could run properly on Windows. The infrastructure setup was done using Python 3.13.5, Apache Spark 4.1.1, PySpark, Java JDK 17, PostgreSQL JDBC driver and Hadoop Windows binaries such as winutils.exe and Hadoop.dll. Several environment variables including SPARK_HOME, HADOOP_HOME and JAVA_HOME were also configured so that Spark could correctly locate the required libraries and dependencies during execution. A significant challenge encountered during this stage was that Spark 4.1.1 was built for Hadoop 3.4.2, for which has no existed at the time of the project. However, the Hadoop 3.3.6 binaries were used as a compatible alternative and the Hadoop.dll file was specially required to enable Spark's ability to write output files on the Windows filesystem. Without this binary, Spark could read data successfully but was unable to commit any written output, resulting in UnsatisfiedLinkError exceptions that had to be systematically diagnosed and resolved before process the pipeline.

Docker and Docker Compose were used to containerize the supporting services to standardize the deployment environment and ensure reproducibility. A docker-compose.yml file was created to manage and orchestrate three services simultaneously. Then, setup a PostgreSQL container named postgres_censo running on port 5432 to serve as the data warehouse. An Adminer container accessible on port 8081 to manage database and visualize schema and a Metabase container running on port 3000 to create dashboard and visualize business intelligence. Hence, all three containers were started together using the docker-compose up command. This allowed to reproduce the entire infrastructure across different machines without manual service configuration.

During this stage, an important lesson about containers within the same Docker Compose network was learned. Containers within the same Docker Compose network communicate using their service names rather than localhost, which means Adminer and Metabase are connected to PostgreSQL using the service name postgres instead of using the local IP address.

3.2 Raw Data Collection and Extraction

The raw data used in the project was obtained from Censo Escolar published by Brazil's National Institute for Educational Studies and Research (INEP) through its open data portal. Each dataset from 2010 to 2021 was provided as individual compressed ZIP file involving 12 files with total extracted size of about 2.65GB. To simplify the extraction process, a Python script named `1_download.py` was developed to automate the decompression and file organization tasks. The script extracted each ZIP archive into its corresponding CSV file and stored all datasets in a centralized data directory at `C:\etl_spark\data`. This automation approach provided better reproducibility and decreased the chance of human error when manually managing 12 large files. Each extracted CSV file contained approximately 370 columns consisting of different school-related information such as student enrollment data, teacher statistics, infrastructure facilities, geographic details and demographic information.



```
1 import os
2 import zipfile
3 import shutil
4
5 DATA_FOLDER = "C:/etl_spark/data"
6 YEARS = [str(y) for y in range(2010, 2022)] # 2010 to 2021
7
8 def extract_file(year):
9     zip_path = f"{DATA_FOLDER}/microdados_censo_escolar_{year}.zip"
10    temp_folder = f"{DATA_FOLDER}/{year}_temp"
11    csv_dest = f"{DATA_FOLDER}/{year}.csv"
12
13    print(f"Extracting {year}...")
14    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
15        zip_ref.extractall(temp_folder)
16
17    # Find the CSV file inside extracted folder
18    found = False
19    for root, dirs, files in os.walk(temp_folder):
20        for file in files:
21            if file.upper().endswith(".CSV"):
22                src = os.path.join(root, file)
23                os.rename(src, csv_dest)
24                print(f"✅ {year}.csv ready!")
25                found = True
26                break
27    if found:
28        break
```

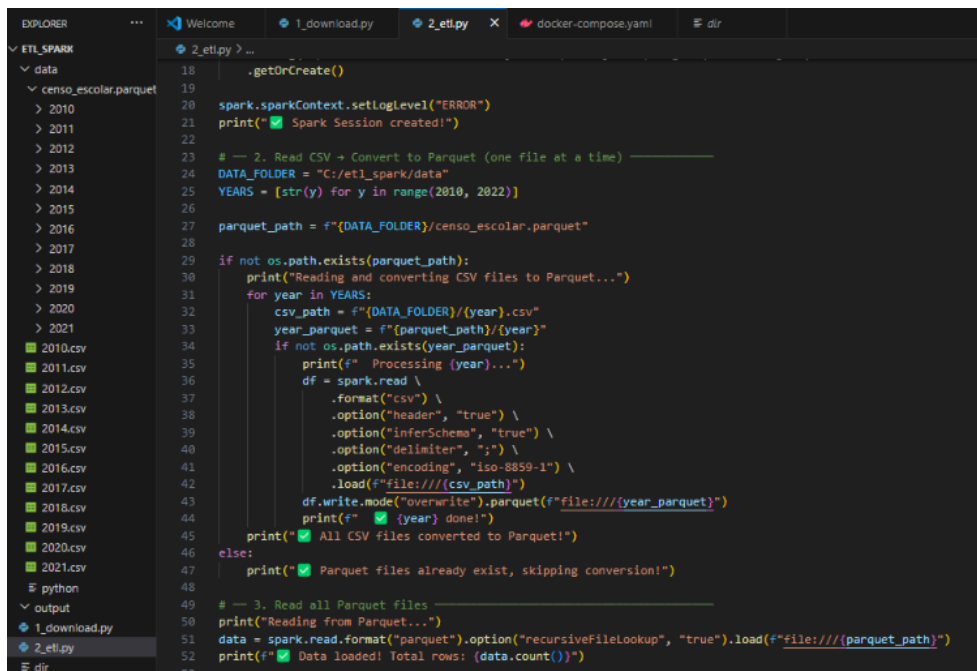
Figure 2: Code Extract Zip Archive to CSV file

3.3 Data Preprocessing

The raw census datasets required several preprocessing steps before they could be processed efficiently using Apache Spark. Each CSV file used semicolons as field delimiters instead of the standard comma, while the character encoding followed the Latin-1 standard due to the dataset contained Portuguese language characters with accented vowels. During ingestion, it was found that recent versions of PySpark no longer recognized the `latin1` alias. Therefore, the encoding was changed to the formal charset identifier `iso-8859-1` to avoid an `IllegalArgumentExpection` during data loading. In addition, the datasets also demonstrated inconsistent schemas across different census years, where certain columns available in later files were missing in earlier ones. Hence, Spark's `inferSchema` option was used to automatically detect the column data types in millions of records, eliminating the need to manually define schemas for every yearly file.

3.4 Data Transformation from CSV to Parquet

After preprocessing the raw CSV files, the datasets were then converted from CSV format into Apache Parquet format. It is a columnar binary file format optimized for analytical workloads. Parquet was selected because it can significantly reduce storage usage through efficient compression while also improving query performance. Unlike CSV files that require scanning entire rows, Parquet stores data by column, allowing quick queries to read only the specific columns and reducing processing time as well as disk usage. Besides, parquet also integrates efficiently with Spark's execution engine for faster data loading and transformation. Therefore, the conversion was implemented within the 2_etl.py script based on year, meaning each CSV files was individually read and written to its own Parquet subfolder under C:\etl_spark\data\censo_escolar.parquet. Once all yearly files were converted, Spark will read all Parquet subfolders together recursively as a single unified dataset for the subsequent transformation and loading stages. This significantly improved processing efficiency compared to read raw CSV files repeatedly.



```
18 .getOrCreate()
19
20 spark.sparkContext.setLogLevel("ERROR")
21 print("✅ Spark Session created!")
22
23 # -- 2. Read CSV + Convert to Parquet (one file at a time) -----
24 DATA_FOLDER = "C:/etl_spark/data"
25 YEARS = [str(y) for y in range(2010, 2022)]
26
27 parquet_path = f"{DATA_FOLDER}/censo_escolar.parquet"
28
29 if not os.path.exists(parquet_path):
30     print("Reading and converting CSV files to Parquet...")
31     for year in YEARS:
32         csv_path = f"{DATA_FOLDER}/{year}.csv"
33         year_parquet = f"{parquet_path}/{year}"
34         if not os.path.exists(year_parquet):
35             print(f"Processing {year}...")
36             df = spark.read \
37                 .format("csv") \
38                 .option("header", "true") \
39                 .option("inferSchema", "true") \
40                 .option("delimiter", ";") \
41                 .option("encoding", "iso-8859-1") \
42                 .load(f"file://{csv_path}")
43             df.write.mode("overwrite").parquet(f"file://{year_parquet}")
44             print(f"✅ {year} done!")
45     print("✅ All CSV files converted to Parquet!")
46 else:
47     print("✅ Parquet files already exist, skipping conversion!")
48
49 # -- 3. Read all Parquet files -----
50 print("Reading from Parquet...")
51 data = spark.read.format("parquet").option("recursiveFileLookup", "true").load(f"file://{parquet_path}")
52 print(f"✅ Data loaded! Total rows: {data.count()}")
```

Figure 3: Code Transform from CSV File to Parquet

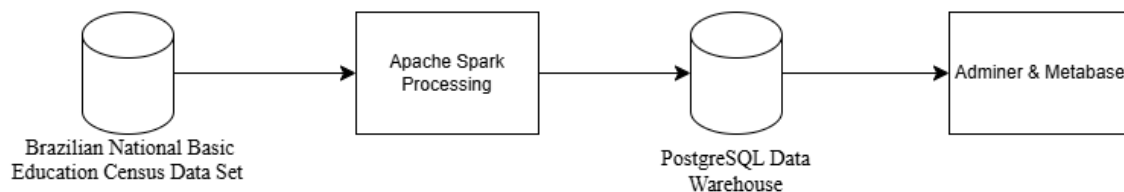
4.0 Modeling

4.1 ETL/Spark Architecture

4.1.1 Architecture Overview

The ETL architecture used in this tutorial has been designed to efficiently process huge amounts of educational census data and map it into a structured analytical database that can be used for business intelligence applications. The raw data is the combination of several annual CSV files from the Brazilian National Basic Education Census from 2010 to 2021, making it important to have a scalable and performance-based architecture to process the vast amount of data.

The architecture does not store raw CSV files directly in the database, but rather applies a structured Extract, Transform and Load (ETL) process to make sure that the data is cleaned, optimized and organized for analytical purposes. The overall flow starts with data extraction from the Brazilian Government Open Data Portal, then data transformation with Apache Spark, and finally, the data is stored in PostgreSQL to be used for reporting and visualization. The architecture used in this tutorial can be summarized into the following flow:



In this architecture, Apache Spark is the main ETL engine, which is used for large-scale data processing; PostgreSQL is the data warehouse that stores structured star schema tables, while Adminer is used for database inspection and Metabase is used for dashboard visualization.

This architecture was chosen to provide an end-to-end data engineering workflow from data acquisition to final reporting for analysis. This is similar to actual business intelligence systems where data needs to be processed and structured before it can be used to draw meaningful insights.

4.1.2 Role of Apache Spark in the architecture

In this ETL pipeline, Apache Spark is the main processing framework. It was an ideal choice to use Spark as the dataset having more than 10 years of data with large size in each file. A dataset like this can consume too much memory and take too long to process using conventional tools like spreadsheets or Pandas.

The distributed processing capability is one of the major strengths of Apache Spark that is demonstrated in this tutorial. Spark distributes computational tasks among multiple execution

units rather than processing data sequentially, which enables the processing of large data sets more efficiently. This project was implemented on a local machine, but Spark still achieved better processing performance by parallel processing and optimizing query planning.

Also, using Spark's lazy evaluation mechanism was a key factor in performance. Unlike Pandas, Spark does not execute operations as soon as the code is written. Instead, it stores all the transformation instructions and constructs an optimized execution plan, only executing the data when an action like writing to storage or displaying results is performed. This will reduce the amount of unnecessary memory used and greatly increase efficiency with millions of records.

The ability to support efficient data transformation and integration is another important aspect of Spark in the architecture. Spark was used to read raw CSV files, automatically infer the schema types, deal with special character encoding, convert CSV to Parquet, and construct fact tables with join operations. If there was no Spark, the processing of multiple years of large-scale census information would have been much more challenging and time-consuming. As a whole, Apache Spark serves as the foundation of the architecture, enabling the ETL process to be scalable, efficient, and capable of processing large analytical datasets.

4.2 Multidimensional Data Model

Once the Etl pipeline got the data processed and loaded in to PostgreSQL, the outcome is a star schema. It is a type of multidimensional data model frequently used in data warehousing. A star schema structures data into one central fact table surrounded by multiple dimension tables. This allows analytical queries to be much quicker and more familiar, compared to the raw, denormalized data.

4.2.1 Star Schema Overview

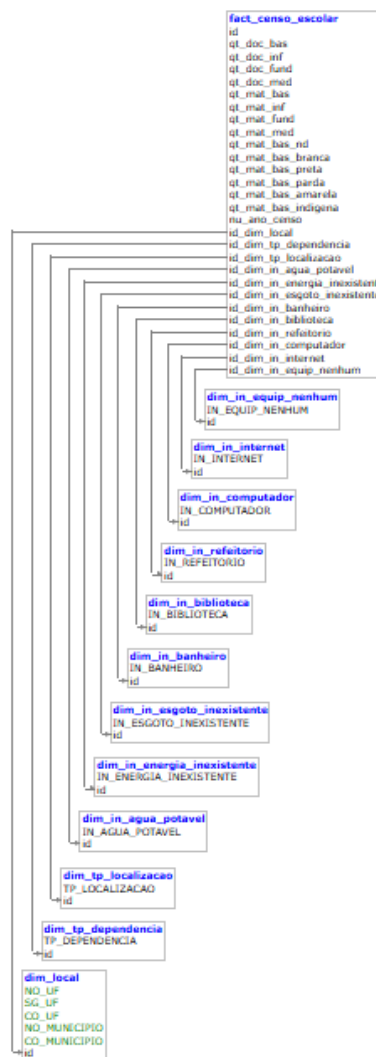


Figure 4: Database Star Schema

The star schema as shown in Figure 2, which we created has one fact table and twelve dimension tables. The fact table is on the top and contains all of the numerical measurement attributes we want to analyze, which are enrollment counts, teacher counts and census year. Every dimension table stands for one descriptive category – like location, school type or an infrastructure attribute and links back to the fact table through the foreign key.

Fact Table: fact_censo_escolar

The fact table is the heart of the star schema. It was made by Apache Spark reading the Parquet file (translated from the raw CSVs), picking relevant measurement columns, joining each dimension table to grab the surrogate key (id) and followed by PostgreSQL write about 3 million rows in to via JDBC connector.

Column	Data Type	Description
id	SERIAL (PK)	Auto-generated primary key
qt_doc_bas	INTEGER	Number of basic education teachers
qt_doc_inf	INTEGER	Number of infant education teachers
qt_doc_fund	INTEGER	Number of elementary education teachers
qt_doc_med	INTEGER	Number of secondary education teachers
qt_mat_bas	INTEGER	Total basic education enrollments
qt_mat_inf	INTEGER	Total infant education enrollments
qt_mat_fund	INTEGER	Total elementary education enrollments
qt_mat_med	INTEGER	Total secondary education enrollments
qt_mat_bas_branca / preta / parda / indigena / amarela / nd	INTEGER	Enrollments broken down by race/ethnicity
nu_ano_censo	INTEGER	Census year (2010-2021)
id_dim_local	BIGINT (FK)	Foreign key to dim_local
id_dim_tp_dependencia	BIGINT (FK)	Foreign key to dim_tp_dependencia
id_dim_tp_localizacao	BIGINT (FK)	Foreign key to dim_tp_localizacao
id_dim_in_* (9 columns)	BIGINT (FK)	Foreign keys to each infrastructure dimension table

Dimension Tables

Each dimension table was made by Apache Spark choosing distinct values of that attribute from the full dataset, taking a surrogate key by using `monotonically_increasing_id()` and writing the outcome to PostgreSQL. Then, a primary key constraint was added from `psycopg2`. There's twelve dimension tables in total:

Table Name	Source Column(s)	Description
dim_local	NO_UF, SG_UF, CO_UF, NO_MUNICIPIO, CO_MUNICIPIO	Geographic location — state and municipality

dim_tp_dependencia	TP_DEPENDENCIA	Administrative type (Federal/State/Municipal/Private)
dim_tp_localizacao	TP_LOCALIZACAO	Urban or rural school location
dim_in_agua_potavel	IN_AGUA_POTAVEL	Whether school has clean drinking water (0/1)
dim_in_energia_inexistente	IN_ENERGIA_INEXISTENTE	Whether school has no electricity (0/1)
dim_in_esgoto_inexistente	IN_ESGOTO_INEXISTENTE	Whether school has no sewage system (0/1)
dim_in_banheiro	IN_BANHEIRO	Whether school has bathrooms (0/1)
dim_in_biblioteca	IN_BIBLIOTECA	Whether school has a library (0/1)
dim_in_refeitório	IN_REFEITORIO	Whether school has a cafeteria (0/1)
dim_in_computador	IN_COMPUTADOR	Whether school has computers (0/1)
dim_in_internet	IN_INTERNET	Whether school has internet access (0/1)
dim_in_equip_nenhum	IN_EQUIP_NENHUM	Whether school has no equipment at all (0/1)

4.2.2 How the Star Schema Works

To query the star schema, join the fact table with one or more dimension tables, through foreign key columns. For instance, to find total enrollments by state and year, join fact_censo_escolar to dim_local using `id_dim_local = dim_local.id`, and then group by the state name and census year. This type of query performs much faster than scanning the original raw CSV file, because the data is pre-aggregated into a structured, indexed format with in PostgreSQL.

The structure allows for slicing and dicing over multiple dimensions at the same time. For example, we could filter for urban school only (via `dim_tp_localizacao`), in a specific state (via `dim_local`), on how enrollment changed from 2010 to 2021 – all in one single SQL query, that joins the fact table to three dimension tables all at once.

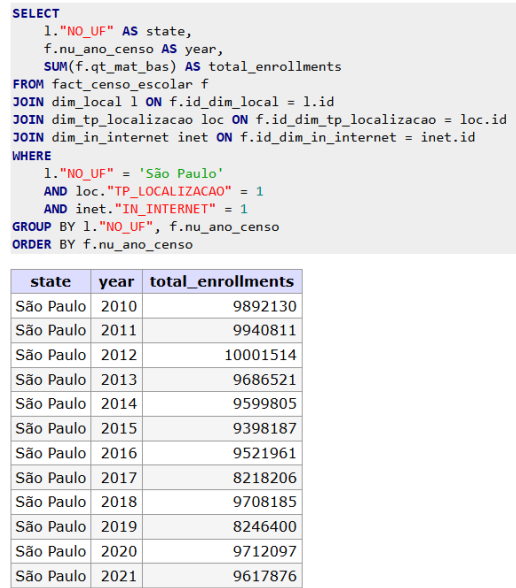


Figure 5: Example SQL command in Adminer

This is precisely what the Metabase dashboard tool was configured to show – connecting Metabase to the censo_escolar PostgreSQL database allows non-technical users to visually explore the trend of enrollment, compare availability of infrastructure between regions and create charts without ever having to write a single line of SQL.

5.0 Evaluation

The evaluation phase assessed whether the ETL pipeline successfully achieved the project's technical and analytical objectives. The pipeline was evaluated across three key dimensions which are processing performance and data integrity, star schema correctness and dashboard visualization effectiveness.

5.1 ETL Pipeline Performance and Data Integrity

The ETL pipeline successfully demonstrated Apache Spark's capability to process a large-scale of data that would be impractical to handle using conventional single-machine tools such as Pandas or standard Python scripts. In this project, the structured PostgreSQL warehouse was able to ingest, convert and load more than 2.65 terabytes of raw census data from 2010 to 2021, which is 12 annual CSV files. The pipeline processed a total of 2,792,984 records that represents individual school-level censuses observations across all 12 years of data.

Spark's lazy execution mechanism that mentioned in data modeling played an essential role in improving the efficiency of ETL pipeline. This is because Spark delays transformation operations such as filtering, selecting and joining until an action is triggered which is different from Pandas who process data immediately after each operation is executed. Instead of processing every single step separately, Spark will build an optimized execution plan before carrying out the actual computation. It helps reduce unnecessary memory usage and repeated processing. Therefore, actions such as writing Parquet files and counting rows triggered Spark to execute the transformations defined earlier in the workflow. In addition, Spark session was configured using local [2], which allows two CPU cores on the local machine to execute tasks simultaneously rather than sequentially. This parallel processing capability enhances the overall performance and enabled the pipeline to handle millions of census records more efficiently.

The conversion of raw CSV files into Apache Spark Parquet format was one of the most significant performance enhancements applied in this pipeline. The Parquet's columnar storage format allowed Spark to execute faster query by reading only the specific columns for each transformation and reduces the storage usage. Each CSV file was individually converted into its corresponding Parquet subfolder on a per-year basis. Spark then recursively read all of the subfolders as a single, cohesive dataset for further transformation and loading phases. Data integrity was validated by confirming that the final record count of 2,792,984 rows in the PostgreSQL fact table matched the source Parquet files. Besides, the Portuguese language characters in state and municipality name fields were correctly preserved without any encoding corruption throughout the pipeline in the data preparation step.

5.2 Dashboard Visualization and Business Objective Evaluation

The processed data was connected to Metabase for dashboard creation and business intelligence visualization. A dashboard named Brazilian education Census dashboard was created and it contains three analytical charts.

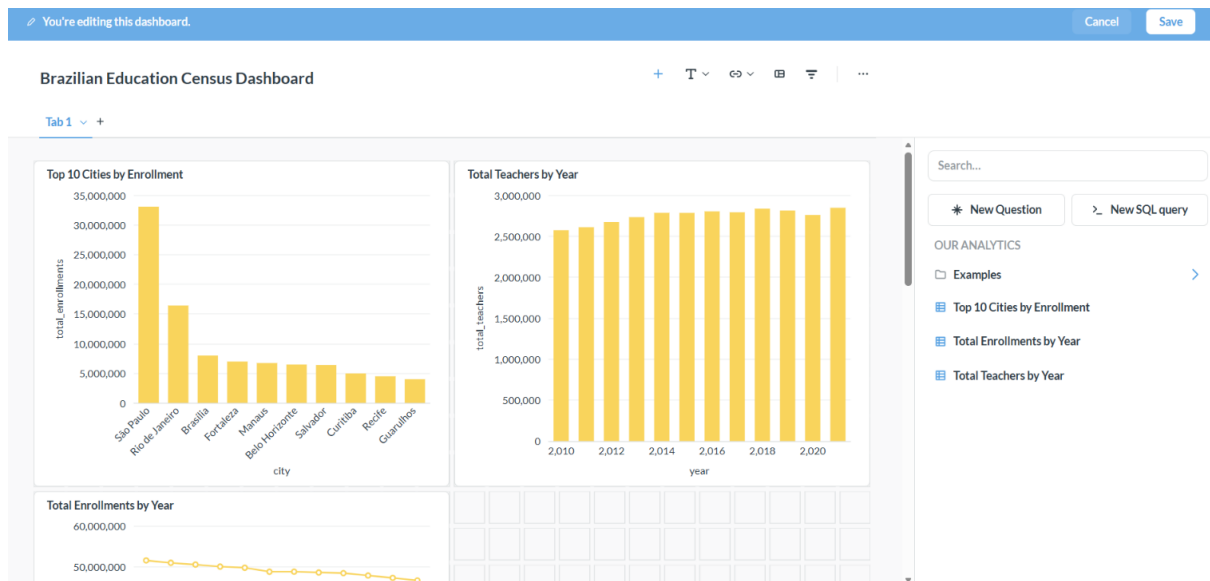


Figure 6: Brazilian Education Census Dashboard

The first chart displayed the total number of school enrollments by year from 2010 to 2021 using a line chart. This visualization revealed a consistent declining trend in the total school enrollments across Brazil throughout the 12 years. This downward trend reflects a well-documented real-world demographic pattern in Brazil, where declining birth rates and increasingly urbanization have gradually affected the number of school-age children entering the education system over time. This finding directly addressed the business objective of visualizing student population changes over year in Brazil and it also demonstrated that the pipeline has successfully captured a meaningful trend from the raw census data.

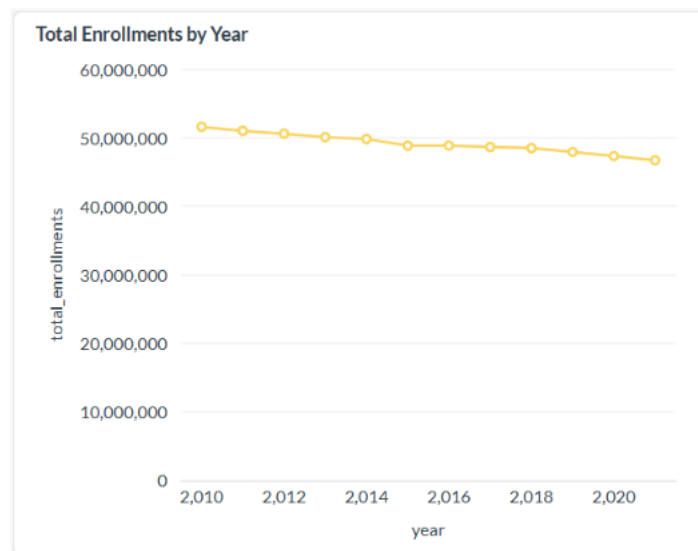


Figure 7: The Line Chart of Total Enrollment by Year

The second chart showed the top 10 cities with the highest total school enrollment using a bar chart. São Paulo ranked consistently at the top as the city with the highest enrollment figures, followed by other cities. This visualization addressed the business problem of finding the best cities based on student population. It also allowed for a direct comparison of cities and showed key regional differences in demand for education around the country. Therefore, the government may take action to improve educational infrastructure and resources for those in lower-enrolment cities.

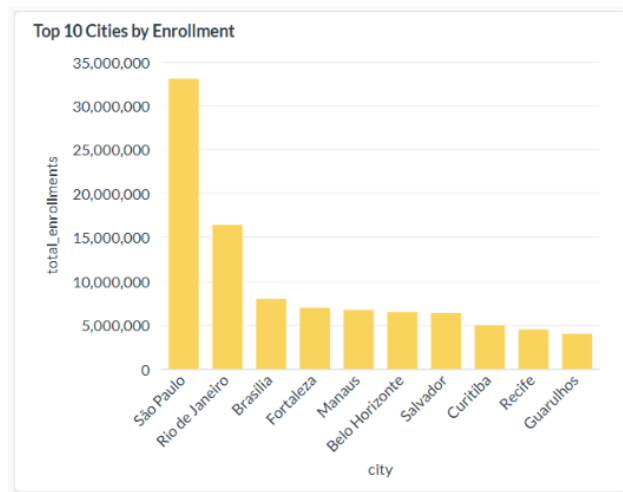


Figure 8: The Bar Chart of Top 10 Cities by Enrollment

The third chart displayed the total number of teachers by year from 2010 to 2021 using a bar chart. This chart complemented the enrollment trend visualization by showing how the teacher availability changed over the same years. Comparing the teacher count trend against the enrollment trend in chart 1, it provides valuable insights into whether teacher staffing levels were proportionally adjusted in response to the declining student population. It then supports more informed decisions around educational workforce planning across Brazil.

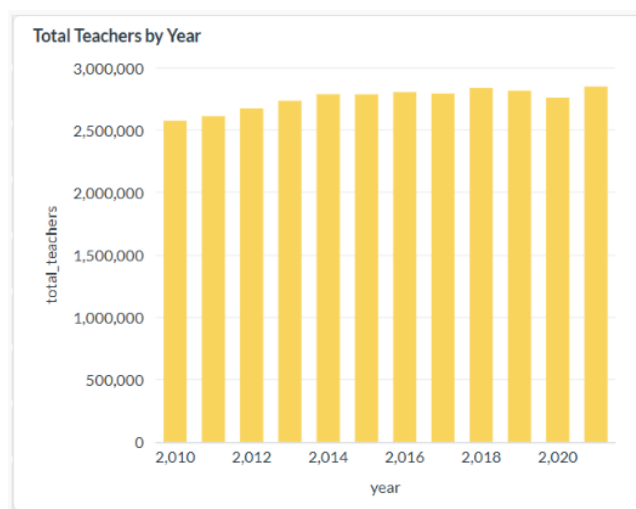


Figure 9: The Bar Chart of Total Teachers by Year

All three charts successfully retrieved data from the PostgreSQL database, rendered correctly within the Metabase interface and reflected trends that were logically consistent with real-world expectations. The dashboard confirmed that the ETL pipeline produced a data warehouse that was not only technically correct but also practically provide useful insights for business purposes. Besides, dashboard could also help non-technical users to explore and understand the educational trends interactively without requiring any SQL or Apache Spark knowledge.

5.3 Limitations, Improvements and Overall Evaluation

Despite the overall success of the ETL pipeline, there were several limitations encountered throughout the project. Since the entire pipeline was run on the local Windows system, Spark was only configured with local[2] meaning that only two CPU cores were used simultaneously instead of a real distributed cluster. This will significantly limit the processing speed and scalability compared to the real results that could achieved on a proper multi-node environment.

Looking ahead, deploying the pipeline on a cloud platform such as Databricks would allow Spark to fully utilize its distributed computing capabilities across multiple clusters. Furthermore, the pipeline is more efficient by only implementing incremental loading, where it only processes newly added data rather than repeating the full batch every run.

Overall, the Apache Spark ETL project successfully met all business objectives set out at the beginning. Apache Spark proved to be a powerful and practical choice for handling a large-amount of raw data compare to Pandas, transforming it into a well-structured star schema and making it accessible through the interactive Metabase dashboards. s

6.0 Reflection

Gui Kah Sin

At the beginning of this tutorial, I faced several challenges while setting up the Apache Spark environment because the provided information was limited and most setup examples were written for Linux instead of Windows. I spent quite some time troubleshooting issues related to Spark, Hadoop and environment variables before successfully configuring everything. However, once the setup was completed, I found the ETL process very interesting because large datasets could be extracted, transformed and processed efficiently through simple command-line execution. Through this project, I learned how to convert data into different file formats such as Parquet, either by processing files individually or all together depending on the system requirements. This was also my first experience using Adminer, Metabase and Docker Compose. Initially, I started each Docker service manually, but later learned how to manage everything more efficiently using a single docker-compose up command. Overall, this tutorial exposed me to many new data engineering tools and concepts, and I believe the knowledge gained will be useful for my future career.

Sabrina Heng Wei Qi

In this tutorial, I had the opportunity to learn and gain valuable hands-on experience in developing a full-fledged ETL pipeline with Apache Spark and enhanced my technical competency and teamwork in data engineering. For this, I had to overcome some challenges in the implementation process, as it was the first time I used technologies like Apache Spark, Docker PostgreSQL, Adminer and Metabase. The first problem was that I had already installed PostgreSQL on my machine and the tutorial was to run the database inside Docker, which led to port conflicts on the default port. I have tried to troubleshoot Docker containers, checked what services are running and how to improve the networking between applications, and I have failed at all these multiple attempts. I had some compatibility problems as well: Some Java versions caused Spark session errors, so I had to do some research to find compatible versions and try out different versions and configurations until I was able to solve the problem. Although these problems were frustrating at first, they taught me that data engineering involves not only coding, but also environment setup, debugging and system integration. As far as teamwork, we always communicated and shared ideas on how to solve problems and discussed technical issues and made sure that the implementation was consistent with the report writing. It was a great experience to learn how to communicate and coordinate with others in a technical project. Technically, I learned how to create ETL workflows, how Apache Spark works with distributed processing, lazy evaluation, how to manage databases using PostgreSQL and how to visualize the data in dashboards with Metabase.

Woo Cheng Shuan

From this tutorial, I got to know a few new tools in data engineering workflow, which are Apache Spark, PostgreSQL (Docker), Metabase and Adminer. I faced challenges in setting up the environment at the beginning, but luckily I overcame them. This project gave us hands-on experience using Apache Spark to process data in parallel across multiple cores. This is because it could handle millions of rows in just a few minutes. I also learned about Docker networking, where containers on different networks can't communicate and container data is lost if volumes aren't properly configured. Moreover, I think the star schema design was particularly interesting because it showed us the difference between how data is stored for operations versus how it is stored for analysis through fact and dimension tables. I also used Adminer to run SQL queries and confirm whether the data is correct or not, and connected Metabase to build visual dashboards. In a nut shell, this tutorial connected many concepts from the course, which are distributed computing, data modelling, database management and visualization, into one complete working pipeline.